

南京邮电大学专业学位硕士研究生学位论文开题报告

学号	1221045617	姓名	邱华杰	手机	15582878382
专业（领域）	电子信息		所在学院	计算机学院、软件学院、网络空间安全学院	
实践企业	无				
已获得的课程学分	29		是否达到培养计划要求	是	
未完成的课程及预计完成时间	无				
补修课程及成绩	无				
初定论文题目	基于深度强化学习的边缘计算迁移与缓存研究				
论文选题来源	中央、国家各部门项目				
论文类型	应用研究				
论文形式	应用技术研究论文				

一、选题依据（综述报告）

（与选题有关的国内外概况和发展趋势，阐述该选题的研究意义，或工程设计的价值和意义，选题的先进性和实用性，技术难度及工作量；列出主要阅读参考文献,不小于 15 篇）

1.1 国内外概况和发展趋势

随着 5G 网络的大规模部署，推动了物联网（Internet of Things, IoT）产业^[1]的变革与发展，不仅可为物联网通信提供超低时延、高可靠、高速率的需求，更有潜力满足用户对服务质量多样化的需求。

物联网已被视为第五代、5G 之后和未来第六代通信系统的主要研究场景，引起工业界和学术界的极大关注。自物联网概念被提出，万物互联的理念深入到社会的各行各业之中，各行业的应用与兴起，在丰富了人民的日常生活、推动了社会发展的同时，用户设备数量的爆炸式增长也带来了网络拥堵^[2]、计算时延滞后的不良影响，造成用户体验质量下降、行业生产安全的隐患。传统云计算在数据高速传输、为设备提供无限的计算资源方面是有效的，但缺乏覆盖性及邻近性，无法满足某些特定场景对上下文高感知能力和低时延的需求，如何快速、高效地对用户任务进行接收、处理成为行业发展新的瓶颈。而边缘计算，以其计算资源下沉至设备边缘、部署灵活等特点，能够更加有效的处理用户设备任务请求^[3]。

移动边缘计算（Mobile edge computing, MEC）通过在网络边缘部署一些 MEC 服务器（MEC servers, MECs），形成一个网络计算、存储、应用核心能力为一体，靠近智能物联网终端设备的 MEC 平台^[4]。该平台可处理大规模数据，提供高效的近端服务，满足智慧城市在实时响应、快速部署、高速运行、安全性等方面的需求。然而，MECs 有限的计算能力并不能完全满足与日俱增的用户需求，MEC 无法替代云计算，而是和终端一起作为云计算的补充，为大数据、物联网等相关应用提供一个高效的计算平台。因此，在智慧城市场景中，学术界和业界将 SMDs、边缘平台和云平台集成为云边端网络架构。该体系结构将数据的计算处理能力下沉到靠近用户的边缘侧和终端，根据具体的 QoS（Quality of service, QoS）需求、设备的计算和存储能力以及网络的综合成本考虑将任务迁移到 SMDs、边缘平台或云平台上。该架构可以充分利用物理设备和设施，有效缓解网络的存储、传输和计算压力，降低端到端时延，提供区分服务的 QoS。由于更加靠近用户，云边端协同网络架构为智慧城市的应用提供更快的响应，将任务需求尽可能地在边缘和终端解决，大大提升处理效率，减轻云端负荷。

边缘计算的发展给物联网带来了许多益处，其靠近用户终端的特性使其可以拥有较低的延迟。同时，大部分流量不用在网络中传输，在数据源处就可以进行处理，有效缓解了网络拥塞。但随着用户终端的爆炸式增长，边缘计算面临着边缘节点资源受限的挑战。为了解决上述挑战，基于边缘计算的计算迁移技术受到了学者们的重视，而如何给出最优的迁移决策并合理分配资源，也成为了该研究领域的一个热点^[5]。

随着边缘计算中计算迁移技术的快速发展，也萌生了许多挑战。这些挑战主要来自于不同的计算迁移场景，不同的环境对于迁移过程的通信和计算成本都有不同的要求，因此，如何满足各个环境下对于计算迁移的不同要求就显得尤为重要。同时，大量用户设备的任务请求有着极大的重复请求概率，因此将部分任务请求结果/内容缓存到存储器中，可以快速将用户任务请求结果反馈给用户设备，极大减少任务处理时延、缓解边缘计算压力，为毫秒级物联网设备需求响应开辟了另一条路线。

计算迁移(Computation Offloading, CO)已经在计算机科学领域得到广泛的研究，包括系统架构，虚拟机迁移和电源管理。截至目前，已经提出了几种计算迁移的体系结构，如：MAUI, CloneCloud, ThinkAir, CONCERT, 和 TROPIC 项目中提出的 Femto-Cloud^[6]。这些问题可以由基于支持分布式的移动边缘计算迁移(Mobile Edge Computation Offloading, MECO)的网络架构来克服。由于迁移带来了额外的通信开销，一个关键的技术挑战是如何平衡计算成本和通信成本之间的权衡，以支持具有增强的用户体验的应用程序，例如更低的延迟和能耗，因此，计算迁移的方案得到了广泛的研究^[7]。

为了提高网络性能，一些研究将重点放在完全迁移策略上，以最小化延迟或能耗^[8]。与完全迁移相比，部分迁移更适合延迟要求更严格的应用程序，因为它利用了智能移动设备和云之间的并行性。此外，由于无线网络的带宽有限，因此迁移部分而非全部应用程序更为合理。因此，许多工作都致力于部分迁移。Munoz 等人通过共同优化智能移动设备和云端的上行链路时间、下行链路时间和处理的数据大小，最大限度地降低了智能移动设备的能耗^[9]。Huang 等人提出了一种基于李亚普诺夫(Lyapunov)优化的动态迁移算法，以实现节能^[10]。Lorenzo 等人联合优化了传输功率和调制方式，以最小化延迟约束下智能移动设备的能量消耗^[11]。Yang 等人联合研究了云资源上：计算的划分和迁移计算的调度，以实现所有用户的最小平均完成时间^[12]。Chen 等人提出了一个划分和执行数据流应用程序的框架，以实现最大速度^[13]。

然而传统的边缘计算迁移算法一般为确定性算法，但面临着 5G 网络环境下呈指数级增长的用户终端及不断变化的网络状态，传统算法显得尤为笨重和力不从心。近年来，随着人工智能的兴起，基于深度强化学习的迁移算法可以通过大数据进行预测，或通过以往的迁移策略进行学习，使得边缘节点的迁移策略能够自主适应各个环境，能够用少量资源得到最优的迁移决策，极大程度上降低了边缘计算的通信和计算成本。Huang 等人^[14]研究多个用户设备将其计算任务迁移到多个 MEC 服务器和一个云服务器，提出了一种基于分布式 DNN 的深度学习算法来求解迁移决策，以最小化用户能耗和时延，但是没有考虑用户的自私性，可能导致各服务器负载不均衡的情况。Chen 等人^[15]考虑了一个随机无线信道和任务到达的多用户多输入多输出(Multiple-Input Multiple-Output, MIMO)系统，为了最小化每个用户的功耗和时延，提出了一种独立式学习的 DDPG 算法，每个用户根据对状态的局部观测来自适应分配设备计算功率和传输功率，其所提出的基于 DDPG 的分配算法可以帮助每个用户学习到有效的动态分配策略，并验证了基于连续动作空间的 DDPG 算法优于基于离散动作空间的强化学习算法 DQN。文献[16]考虑一个多用户大规模 MIMO 的移动边缘网络，提出了一个在严格延迟约束下以用户能耗最小为目标的迁移决策和资源分配决策问题，然后提出了一种基于多智能体深度确定性策略梯度算法(Multi-Agent Deep Deterministic Policy Gradient, MADDPG)的分布式协同求解方法，减轻了集中式算法带来的巨大开销。文献[17]提出了一种基于深度强化学习的无模型分布式算法，每个设备都可以在不知道其他设备的任务模型和卸载决策的情况下确定自己的卸载决策。为了改进算法中对长期代价的估计，引入了长短期记忆(LSTM)、深度 q-网络(DQN)和双 DQN 技术。文献 0 为了解决复杂的随机优化问题，设计了马尔可夫决策过程(MDP)建模，并借助于深度强化学习(DRL)来处理巨大的状态空间。采用参数共享网络结构和卷积神经网络(CNN)相结合的方法对策略函数和价值函数进行近似，有效地提取具有代表性的特征。通过对状态和奖励表示的一系列调整，进一步提高了训练效率。

然而，目前多接入边缘计算迁移问题的研究考虑的因素越来越多也越来越复杂。现有研究多将任务迁移模式集中在整体迁移上，任务模型不够细致，在研究场景上较单一，仅考虑 MEC 服务器与用户设备两层架构，没能充分地考虑用户个性需求，采用算法性能差强人意。少有的针对任务部分迁移的研究也没能

充分利用系统场景资源,存在优化空间。在多用户迁移决策和资源分配决策联合优化的同时,问题的复杂度也将呈指数级增加。因此,如何设计系统的任务模型和场景模型,如何求解高复杂度的优化问题亟待解决。

为了解决上述相关研究存在的问题,提高用户对处理时延、能耗等的实际需求,本课题拟开展以下几点研究内容:设计基于移动用户终端、边缘计算节点、云数据中心的三层计算迁移模型、基于深度强化学习的边缘计算任务迁移研究、基于内容流行度的任务请求缓存研究、基于 Kubernetes 容器编排工具的云计算管理平台研究。本课题的研究具有重要的理论价值和应用前景,能够满足用户对低时延、智能化迁移服务的综合要求。

1.2 研究意义

(1) 边缘计算作为云计算的下沉计算模式,将任务迁移运用到边缘计算当中,以达到任务处理时延和能耗最小化的目的。在 5G 物联网场景下,研究设计一种云边端协同的边缘云计算迁移算法,可以有效降低边缘节点的能耗与时延,提升终端用户体验。

(2) 缓存策略能够存储任务内容、将任务结果快速地传输给用户设备的处理模块,并充分利用缓存空间大小,将其应用于边缘计算建模中,减少终端与边缘节点之间非必要的计算任务数据传输,缓解了大规模物联网设备接入 5G 网络造成的网络传输拥塞的现状。

(3) 研究设计一种基于 Kubernetes 容器编排工具的云计算管理平台,为满足 MEC 场景下,业务的快速部署与云边端协同的业务迁移需求,降低业务迁移部署与动态扩缩容的复杂度。

1.3 选题的先进性和实用性

针对当前 5G 网络环境超高数据传输速率、超低延迟、大规模连接、极低能耗等特点,在 MEC 场景下,通过融合深度强化学习与云边端协同技术,进一步降低边缘计算迁移在各类环境中的任务执行时延和计算成本,从而优化边缘计算迁移与缓存策略的动态自适应性,提高边缘计算迁移的效率。

相比于传统移动云计算(Mobile Cloud Computing, MCC),一方面,MEC 大大降低了数据来回链路的传输时延,同时减少了核心网络的流量负荷,缓解网络拥塞,减轻了核心网络的计算负载。另一方面,MEC 将云计算数据中心的存储、计算等能力下沉到网络的边缘侧,为用户提供计算和存储等资源,从而降低了计算密集型任务的传输时延和计算时延,并且大幅降低了设备的计算能耗^[19]。因此 5G 与 MEC 的融合在面对海量数据流量时,可以进行数据的快速处理,有效解决资源受限的移动设备和计算密集型任务之间的矛盾。由此,MEC 被认为是 5G 中的关键技术之一^[20]。

虽然 MEC 拥有以上诸多优势,但同时也面临着各个方面的挑战。其中最关键的就是关于任务的计算迁移问题。边缘计算将网络边缘上的计算、存储等资源进行有机融合,构建起统一的用户服务平台,按规定的服务原则对网络边缘节点任务请求及时响应并有效处理。由于边缘节点计算能力、资源、带宽、能源等受限,计算迁移变得异常重要。计算迁移是边缘计算的一个关键技术^[21]。

计算迁移将计算迁移到资源更丰富的云中,因此需要做出决定,以考虑将哪些部分迁移到云中。为了做出有效的迁移决策,应该考虑两个方面,包括迁移方法和迁移因素。迁移方法主要包括静态和动态,迁移因素包括应用程序划分方案,移动数据流量,应用程序迁移,能源性能,安全性和隐私保护。首先,弹性移动应用程序在移动智能设备上执行,如果计算资源不能满足应用程序的需求,则激活计算迁移机制。其次,根据计算迁移因素,做出相应的迁移决策。

因此,一个高效的迁移与缓存策略可以有效降低用户时延、能耗以及提高 MEC 系统各维度资源的利用率。但是现有移动通信网络日益复杂化,导致无线信道状态呈现出无法预知的高度动态性和随机性。MEC 服务器的计算资源、存储空间也会因各种设备的任务请求而动态变化^[22]。这些高度变化的因素使得迁移与缓存策略的制定变得困难,也正因为这些高度动态变化的因素,要求迁移与缓存策略不能一成不变,需要在动态学习中考虑边缘节点分布相对移动端设备分布不均匀、边缘节点之间的性能差异性等实际因素。从而最大化所有边缘节点的性能,更好地提升用户服务体验。

1.4 技术难度和工作量

(1) 需要深入研究 MEC 环境下边缘计算的任务场景,并面向其设计一种基于移动用户终端、边缘计算节点、云数据中心的三层边缘计算迁移模型。

(2) 需要研究计算迁移机制, 根据计算任务的 QoS 设计优化问题模型, 并基于深度强化学习设计一种以计算时延与能耗为优化目标的云边端协同的边缘计算迁移算法, 降低终端任务计算的成本。

(3) 需要研究基于内容流行度的缓存策略, 将部分任务请求结果/内容缓存到存储空间中, 分析任务缓存的常见指标, 设计内容流行度分析方法, 提出一种基于内容流行度的任务请求缓存策略。

(4) 需要深入研究 Kubernetes 容器编排工具的工作原理, 研究 API Server 与 Scheduler 的实现机制, 实现边缘节点上容器的动态部署迁移及扩缩容。

1.5 参考文献

- [1] Cheng N, Lyu F, Quan W, et al. Space/aerial-assisted computing offloading for IoT applications: A learning-based approach[J]. *Journal on Selected Areas in Communications*, 2019, 37(5): 1117-1129.
- [2] Seid A M, Boateng G O, Anokye S, et al. Collaborative computation offloading and resource allocation in multi-UAV-assisted IoT networks: A deep reinforcement learning approach[J]. *Internet of Things Journal*, 2021, 8(15): 12203-12218.
- [3] 贾淑霞, 郝万明, 高梓涵, 杨守义. 任务迁移的移动边缘计算系统中资源分配和任务卸载策略[J]. *电讯技术*, 2022, 62(08): 1037-1043.
- [4] Li X, Da Xu L. A review of Internet of Things—resource allocation[J]. *Internet of Things Journal*, 2020, 8(11): 8657-8666.
- [5] Persone V D N, Grassi V. Architectural issues for self-adaptive service migration management in mobile edge computing scenarios[C]. *International Conference on Edge Computing*, 2019: 27-29.
- [6] Li Y, Wang S. An energy-aware edge server placement algorithm in mobile edge computing[C]. *International Conference on Edge Computing*, 2018: 66-73.
- [7] Sardellitti S, Scutari G, Barbarossa S. Joint optimization of radio and computational resources for multicell mobile-edge computing[J]. *Transactions on Signal and Information Processing over Networks*, 2015, 1(2): 89-103.
- [8] Barbarossa S, Sardellitti S, Di Lorenzo P. Joint allocation of computation and communication resources in multiuser mobile cloud computing[C]. *14th workshop on signal processing advances in wireless communications*, 2013: 26-30.
- [9] Munoz O, Pascual-Iserte A, Vidal J. Optimization of radio and computational resources for energy efficiency in latency-constrained application offloading[J]. *Transactions on Vehicular Technology*, 2014, 64(10): 4738-4755.
- [10] Huang D, Wang P, Niyato D. A dynamic offloading algorithm for mobile computing[J]. *Transactions on Wireless Communications*, 2012, 11(6): 1991-1995.
- [11] Di Lorenzo P, Barbarossa S, Sardellitti S. Joint optimization of radio resources and code partitioning in mobile edge computing[J]. *arXiv preprint arXiv:1307.3835*, 2013.
- [12] Yang L, Cao J, Cheng H, et al. Multi-user computation partitioning for latency sensitive mobile cloud applications[J]. *Transactions on Computers*, 2014, 64(8): 2253-2266.
- [13] Chen J, Xing H, Xiao Z, et al. A DRL agent for jointly optimizing computation offloading and resource allocation in MEC[J]. *Internet of Things Journal*, 2021, 8(24): 17508-17524.
- [14] Huang L, Feng X, Zhang L, et al. Multi-server multi-user multi-task computation offloading for mobile edge computing networks[J]. *Sensors*, 2019, 19(6): 1446.
- [15] Chen Z, Wang X. Decentralized computation offloading for multi-user mobile edge computing: A deep reinforcement learning approach[J]. *EURASIP Journal on Wireless Communications and Networking*, 2020, 2020(1): 1-21.
- [16] Tilahun F D, Abebe A T, Kang C G. Multi-Agent Reinforcement Learning for Distributed Joint Communication and Computing Resource Allocation over Cell-Free Massive MIMO-enabled Mobile Edge Computing Network[J]. *arXiv preprint arXiv:2201.09057*, 2021.
- [17] Tang M, Wong V W S. Deep reinforcement learning for task offloading in mobile edge computing systems[J].

- IEEE Transactions on Mobile Computing, 2020.
- [18] Zhan W, Luo C, Wang J, et al. Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing[J]. Internet of Things Journal, 2020, 7(6): 5449-5465.
- [19] Giust F, Verin G, Antevski K, et al. MEC deployments in 4G and evolution towards 5G[J]. ETSI White paper, 2018, 24(2018): 1-24.
- [20] Chen J, Xing H, Xiao Z, et al. A DRL agent for jointly optimizing computation offloading and resource allocation in MEC[J]. Internet of Things Journal, 2021, 8(24): 17508-17524.
- [21] 施巍松,孙辉,曹杰,等.边缘计算:万物互联时代新型计算模型[J].计算机研究与发展, 2017, 54(5):907-924.
- [22] Elgendy I A, Zhang W Z, He H, et al. Joint computation offloading and task caching for multi-user and multi-task MEC systems: reinforcement learning-based algorithms[J]. Wireless Networks, 2021, 27(3): 2023-2038.

1. 本报告 A4 纸双面打印，交学院一份，学生自留底稿。

二、课题内容及具体方案

（包括研究内容，拟采用的设计方法，技术路线，实验方案的可行性分析）

2.1 研究内容

（1）研究一种基于深度强化学习的云边端协同的边缘计算迁移算法，在缓存容量大小、任务处理时延的约束条件下，最小化时延和能耗成本，充分利用 MEC 计算资源，实现边缘计算迁移优化策略。

（2）研究一种基于内容流行度的请求任务缓存策略，考虑到大量用户设备对某类任务请求重复性，将此类任务缓存到存储空间中可以大大减少任务处理时延、缓解边缘节点有限计算、带宽资源，在提高缓存命中率的同时可以有效减少降低任务计算迁移的成本。

（3）设计实现基于容器的 MEC 的边缘云计算管理平台，利用 Kubernetes 容器编排工具，实现云边计算任务的动态部署迁移与扩缩容，以及资源管理的自动化。适应物联网边缘应用的需求，实现对时延敏感与计算密集型任务的快速响应。

2.2 设计方法

（1）首先，阅读国内外相关文献，研究移动边缘计算场景下计算迁移与内容缓存的机制和原理，为后续深度强化学习算法设计提供思路。

（2）其次，阅读国内外相关文献，研究深度强化学习模型的调优方式，通过实验数据分析调优方式对模型性能的影响情况。

（3）最后，阅读 Kubernetes 开发文档，学习 API Server 接口文档，通过编码实现对业务容器的迁移控制，并将计算迁移算法应用移植到边缘计算管理平台。

2.3 技术路线

2.3.1 基于深度强化学习的时延和能耗云边端协同的边缘计算迁移算法

计算迁移技术是 MEC 的一种关键技术，具体是指移动设备将任务整体或者部分地迁移到边缘计算服务器处完成的技术，一般情况下，计算迁移场景如图 1 所示。

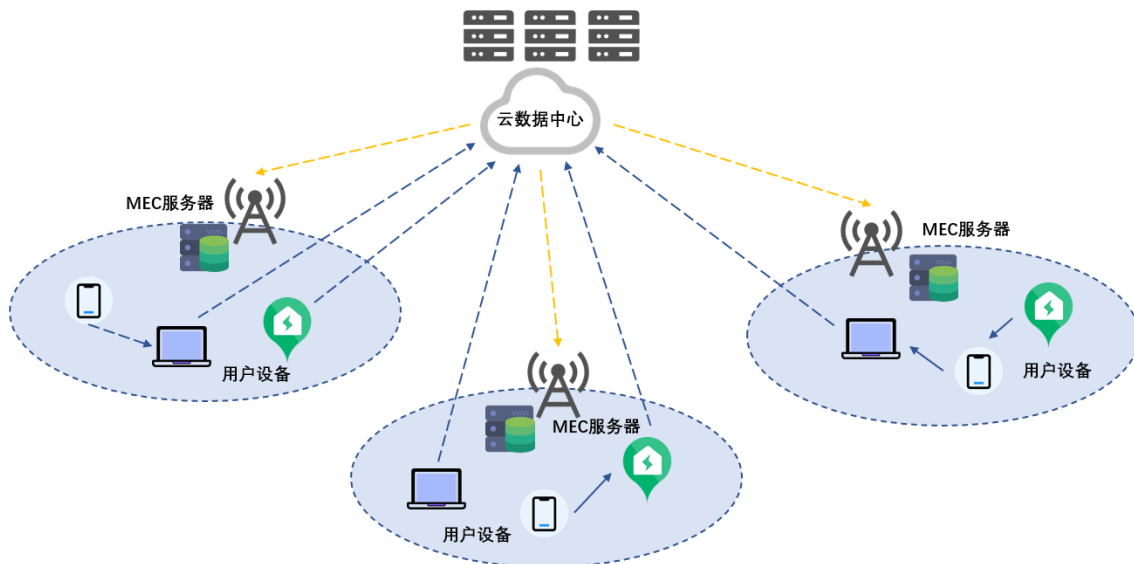


图 1 MEC 计算迁移场景图

任务的完成包括云数据中心迁移计算、MEC 服务器迁移计算、D2D 设备迁移计算三种任务迁移计算模式和任务本地计算模式。MEC 场景中计算迁移技术的提出来源于计算密集型任务与移动设备计算能力和能量有限之间的矛盾，用以满足移动设备在任务完成时对时延与能耗的要求，具体为，通过做出合理的迁移决策将计算任务迁移至算力资源充足的节点，再从该节点处回传计算结果。其核心问题是任务是不是要迁移，迁移到哪处和迁移多少。不同的迁移模式有着不同的任务完成时延和能耗，在计算和

通信资源的限制下，对于如何高效地完成计算任务这一问题，应充分考虑通信系统内各个因素，仔细地规划分析，联合优化迁移决策和资源分配策略，以达到降低任务完成成本，提升迁移效率等目的。这在多接入边缘计算系统中称为计算迁移问题。

DQN 系列强化学习算法中，主要对价值函数进行了近似表示，基于价值来学习。这种 Value Based 强化学习方法在很多领域都得到比较好的应用，但是 Value Based 强化学习方法也有很多局限性，如对连续动作的处理能力不足。同时 Value Based 强化学习对受限状态下的问题处理能力不足，因此一些场景下需要引入基于策略的 Policy Based 强化学习方法。在 Policy Based 强化学习方法下，将策略表示成一个连续的函数后，可以用连续函数的优化方法来寻找最优的策略。

Actor-Critic 就是将 Policy Based 和 Value Based 相结合的方法，Actor-Critic 包括两部分，演员(Actor)和评价者(Critic)。其中 Actor 负责生成动作 (Action) 并和环境交互。而 Critic 使用价值函数，负责评估 Actor 的表现，并指导 Actor 下一阶段的动作。使用类似 DQN 中用的价值函数来替代蒙特卡罗法，作为一个比较通用的 Critic。

基于 Actor-Critic 深度强化学习算法，设计一种云边端协同的边缘云计算迁移算法。算法的具体执行流程如图 2 所示。

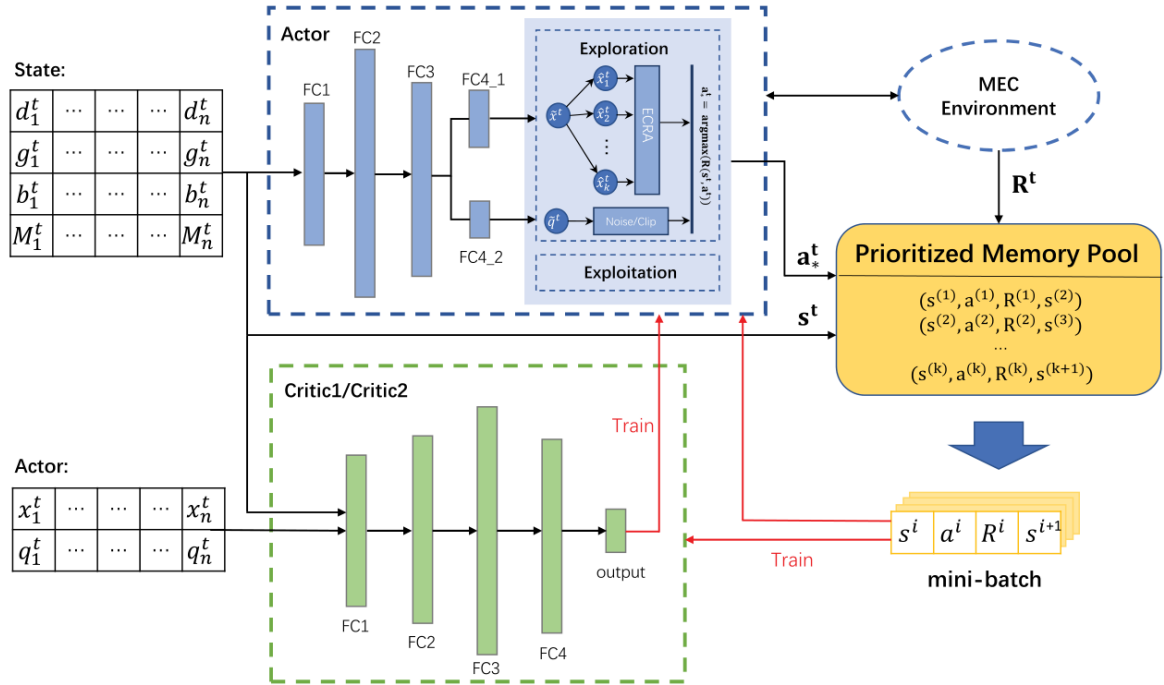


图 2 算法执行流程图

基于 MEC 网络模型定义，假设 i 表示第 i 个用户终端，其中 $i \in \{1, 2, \dots, N\}$ ； j 表示第 j 个边缘节点，其中 $j \in \{0, 1, 2, \dots, M\}$ ，当 $j=0$ 时，特指用户终端本身。每个用户可以将计算任务迁移到指定的边缘节点，边缘节点可为迁移任务分配带宽和计算资源。当多个用户有任务需要计算时，首先确定任务的最大容许延迟，以及本地计算的时间，若本地计算的时间大于最大容许延迟，将发送任务迁移请求，把需要计算的任务数据大小 D_i 发送给边缘节点。计算任务描述为：

$$F_{ij} = \{x_{ij}, \lambda_{ij}, \beta_{ij}, D_i, T_i^{\text{tol}}\} \quad (1)$$

其中 x_{ij} 表示计算任务在本地执行还是被迁移； λ_{ij} 表示边缘节点 j 分配给用户 i 的带宽占比， β_{ij} 表示边缘节点 j 分配给用户 i 的计算资源占比； T_i^{tol} 表示用户终端 i 的最大容许延迟。边缘节点通过在边缘云下同步更新和维护上述任务表格，此同步只需要在边缘节点每次做出任务迁移决策后，更新表格信息，并广播给同一边缘云下的所有边缘节点。因此由全部任务生成的表格可以得到总任务集 F 。

$$F = \{F_{ij} \mid i \in \{1, 2, \dots, N\}, j \in \{0, 1, 2, \dots, M\}\} \quad (2)$$

总任务集 F 将会形成于边缘云中的所有边缘节点。基于总任务集 F ，本文可以得到所有任务的迁移决策，从而计算出各个任务相应的处理成本。

用户 i 的任务迁移到边缘节点 j 的执行过程总延迟为：

$$T_{ij}^{\text{total}} = T_{ij}^{\text{t}_u} + T_{ij}^e + T_{ij}^{\text{t}_d} \quad (3)$$

其中 $T_{ij}^{\text{t}_u}$ 表示在迁移计算中用户 i 上传计算任务到边缘节点 j 的传输延迟当数据传输到边缘节点后， T_{ij}^e 表示利用边缘节点分配给任务的 CPU 资源来进行计算，可以得到用户 i 的任务在边缘节点 j 上的计算时间， $T_{ij}^{\text{t}_d}$ 表示当边缘节点 j 将用户 i 的任务计算完成后，将计算结果返回用户 i ，其中，边缘节点 j 发送结果到用户 i 的传输延迟。

任务 i 迁移到边缘节点 j 过程中用户端消耗的总能耗为：

$$E_{ij}^{\text{total}} = E_{ij}^{\text{t}_u} + E_{ij}^{\text{t}_d} + E_{ij}^w \quad (4)$$

最后结合用户 i 的任务迁移到边缘节点 j 执行过程中的总延迟和总能耗，即公式 (3) 和 (4)，可以得到迁移过程的总成本为：

$$C_{ij}^{\text{total}} = \alpha T_{ij}^{\text{total}} + (1 - \alpha) E_{ij}^{\text{total}} \quad (5)$$

基于上述计算模型描述的系统模型，将时延与能耗作为度量网络性能的两个核心指标，优化目标主要集中在用户层全部任务执行完成时间和能耗上，针对 MEC 环境下时延敏感和计算密集型任务的计算迁移优化问题模型可以描述为：具体优化目标即为最小化所有用户的任务执行延迟和能耗的权重和，即总成本 C 。方式即为通过联合优化迁移决策、带宽分配和计算资源分配来实现，其中任务可在本地执行或者迁移执行，具体优化问题构建如下：

$$\begin{aligned} \min_{\lambda_{ij}, \beta_{ij}, z_{ij}} C = & \min_{\lambda_{ij}, \beta_{ij}, z_{ij}} \sum_{j=0}^M \sum_{i=1}^N z_{ij} \left[(1 - x_{ij}) C_{ij}^l + x_{ij} C_{ij}^{\text{total}} \right] \\ \text{s.t.} \quad & \begin{cases} (1 - x_{ij}) T_{ij}^l + x_{ij} T_{ij}^{\text{total}} \leq T_i^{\text{tol}}, \\ \sum_{i=1}^N z_{ij} \lambda_{ij} \leq 1, \forall j \in \{0, 1, \dots, M\}, \\ \sum_{i=1}^N z_{ij} \beta_{ij} \leq 1, \forall j \in \{0, 1, \dots, M\}, \\ z_{ij} \in \{0, 1\}. \end{cases} \end{aligned} \quad (6)$$

在动态变化的计算卸载环境中，每个终端根据当前系统状态选择是否将到达时间为 t 的任务卸载到 MEC 服务器。由于 MEC 网络存在高维状态空间和动作空间，通常无法获得系统状态转移，且难以通过动态规划构建，因此提出了一种基于深度强化学习的方法来解决这个问题。这使得可以在不知道 MEC 网络的动态性的情况下获得接近最优的迁移策略。根据上述的系统模型，基于强化学习的方法的关键因素定义如下：

(1) 状态空间：在每个时间段 t ，系统状态空间 s^t 包括终端与边缘节点之间的距离和通道收益，到达任务的数据大小，以及 t 开始时的可用能耗。因此，系统状态空间 s^t 可以定义为：

$$s^t = [d^t, g^t, b^t, M^t] \quad (7)$$

(2) 动作空间： $z_{ij}(t)$ 表示 t 时刻用户 i 选择边缘节点 j 进行迁移， $\lambda_{ij}(t)$ 和 $\beta_{ij}(t)$ 分别表示 t 时刻边缘节点 j 分配给用户 i 的带宽和 CPU 占比。因此，动作空间 A^t 可以定义为：

$$A^t = (z_{ij}(t), \lambda_{ij}(t), \beta_{ij}(t)) \quad (8)$$

(3) 奖励函数：边缘节点智能体将在执行每个可能的动作 a 后，在某个状态 s 中获得奖励值 $R(s, a)$ 。

因为通常奖励函数与目标函数相关，优化问题的目标是最小化所有用户的任务执行延迟和能耗权重和的总成本，而求解目标是获得最大的奖励函数值，因此奖励函数与总成本的大小是负相关的。据此，奖励函数定义为：

$$R(s^t, A^t) = -(C_{total}^{(1)}(s^t, a^t) + C_{i(p)}^t) \quad (9)$$

为了将边缘节点智能体中的各种观测结果融入迁移策略制定的服务中，定义策略函数为：

$$\pi(s_t, a_t) \approx \pi(a_t | s_t; \theta_t) \quad (10)$$

其中 θ 为使用策略迭代更新网络中的权重参数。由于策略函数的目标是最大化奖励，因此可以使用梯度上升算法计算关于奖励的期望 $E[R_t]$ 。策略梯度的更新公式为：

$$\nabla_{\theta} E[R_t] = \nabla_{\theta} \log \pi(a_t | s_t; \theta_t) R_t \quad (11)$$

2.3.2 基于内容流行度的任务请求缓存策略研究

为了克服本地物联网设备资源受限的问题，研究点一引入了计算迁移技术；而为了有效提高网络能效，最大限度利用好边缘节点的缓存空间，又加入了对于内容缓存技术的考量。

内容缓存是指将用户所需的内容或计算任务缓存在服务器中，用户请求若是命中缓存，则直接从缓存中返回内容或任务结果。而边缘内容缓存就是将缓存中心放在网络边缘，即离用户端很近的位置，然后将那些会被频繁访问的网络内容存储在其中，这样就会使得用户的访问速度更快。除了增强用户的体验外，提前存储体积大的流行性文件还可以节省网络带宽，努力到达传输带宽消耗和缓存代价之间的均衡点

但是用户的请求是各种各样的，所以无法提前存储所有的内容，而且边缘节点的存储空间大小是有限的，所以就需要在缓存内容上有所取舍。内容缓存的一大特点就是缓存内容的流行性分析。流行度的定义为文件内容被请求的次数与所有用户总请求的文件次数之间的比例，一般情况下只是在一个特点时间段的特定区域内获得，不然范围太广无法确定。内容缓存应用在边缘计算中是有很多益处的，但是在不同的应用程序中计算任务是有着不同的性能需求的，那么在内容缓存中就有着不一样的衡量指标，主要分为以下几种常见指标：系统整体容量、时延、能耗、缓存内容流行度。

为了在有限的缓存空间中尽最大概率命中缓存，提出一种基于用户推荐的高流行度缓存策略，如图 3 所示。

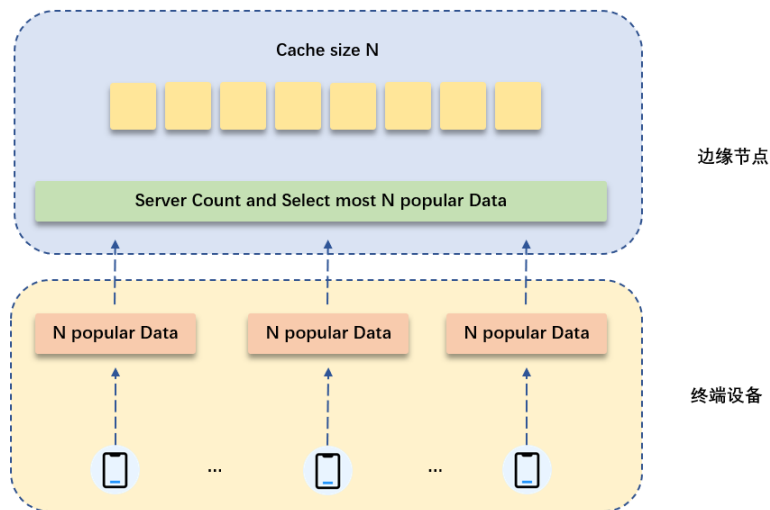


图 3 基于内容流行度的计算迁移缓存策略

通常情况下，内容缓存中有两个问题是重点：内容缓存位置以及缓存内容的传输。前一个问题包括

每一个缓存内容的位置和大小，这是对于数据中心的选择以及边缘节点的选择的问题；后一个问题是怎样将缓存内容以最小代价发送给请求用户。

为解决上述问题，在提出的基于内容流行度的计算迁移缓存策略中，每个时间段 t 服务器会根据提出的缓存替换策略，从多个计算数据中选择部分数据进行预存。在下一个时隙 t 中，如果用户请求的文件已经缓存在服务器中，则不需要计算和卸载这些文件。对于服务器中没有缓存的数据，采用计算迁移算法寻找合适的迁移策略进行计算迁移。

2.3.3 基于容器的 MEC 边缘云计算管理平台的实现

传统云计算是将微服务部署于虚拟机中，OpenStack 提供了云平台的基础设施。边缘计算是云平台的延伸，但缺少云数据中心的高性能服务物理设备来部署和运行完整的虚拟化环境。于是，轻量级的容器取代了虚拟机成了边缘计算平台的标准技术之一。在容器中，网络，内存和文件系统是隔离的。容器引擎管理所有的容器。所有的容器共享物理主机上的操作系统内核。以 Docker 为主的容器技术逐渐成为边缘计算的技术标准，各大云计算厂商都选择容器技术构建边缘计算平台的底层技术栈。通过容器技术进行对资源的隔离，不仅对 CPU、内存和存储的额外开销非常小，而且容器的生命周期也非常迅捷，可以在毫秒级别的时间内开启和关闭容器。然而对大量容器的控制操作是机器复杂繁琐的。因此谷歌公司开发了 Kubernetes 开源容器管理工具，提供高度的互操作性、自我修复、自动升级回滚一级存储编排等功能，已经成为边缘计算的技术标准。

因此为满足 MEC 场景下，业务的快速部署与云边端协同的业务迁移需求，设计实现了一种基于 Kubernetes 容器编排工具的云计算管理平台。平台系统架构如图 4 所示。

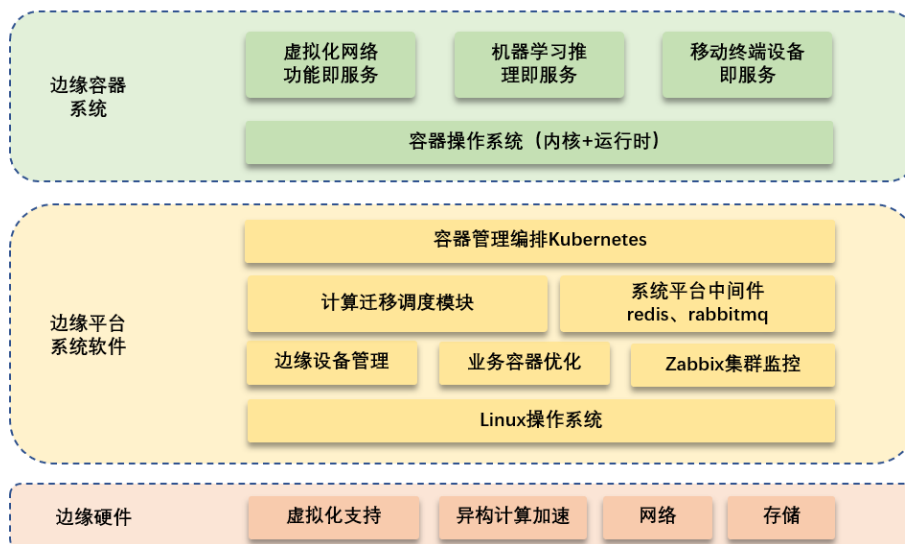


图 4 基于容器的 MEC 的边缘云计算管理平台系统架构

Kubernetes（简称 K8s，其中 8 代指中间的 8 个字符），是一个全新的基于容器技术的分布式架构方案，这个方案虽然还很新，但却是 Google 十几年来大规模应用容器技术的经验积累和升华的重要成果，确切的说 Google 一个久负盛名的内部使用的大规模集群管理系统——Borg 的开源版本，其目的是实现资源管理的自动化以及跨数据中心的资源利用率最大化。Kubernetes 具有完备的集群管理能力，包括多层次的安全防护和准入机制、多租户应用支撑能力、透明的服务注册和服务发现机制、内建的智能负载均衡器、强大的故障发现和自我修复能力、服务滚动升级和在线扩容能力、可扩展的资源自动调度机制，以及多力度的资源配额管理能力。

Kubernetes master 组件包含：（1）API Server 提供 HTTP Rest 接口，是所有资源增删改查和集群控制的唯一入口。可以通过 kubectl 工具来与其交互。API Server 是集群管理的 API 入口和资源配额控制入口，也提供完备的集群安全机制。（2）Controller Manager 资源对象的控制自动化中心。即监控 Node，当故障时转移资源对象，自动修复集群到期望状态。（3）Scheduler 负责 Pod 的调度，调度到最优的 Node。

同时将计算迁移策略的生成模块嵌入系统实现中，利用 Scheduler 组件与 kubectl 交互控制工具，实现将业务容器按照生成的迁移策略执行迁移操作。Kubernetes 与边缘计算的结合可以实现让计算更贴近

设备侧，以便降低时延、减小带宽需求和提高可靠性，满足用户实时、智能、数据聚合和安全需求。使得边缘侧减少与云端的通信，以便对某些应急情况做出快速响应。将数据流应用部署到边缘节点，降低设备和中心云端之间通信的网络时延，降低设备端的能耗。

2.4 实验方案的可行性分析

（1）本研究方案首先针对 MEC 环境下边缘计算的任务场景，研究设计一种基于移动用户终端、边缘计算节点、云数据中心的三层边缘计算迁移模型。其次，研究深度强化学习，设计一种以计算时延与能耗为优化目标的云边端协同的边缘计算迁移算法与任务缓存策略。最后，将计算迁移算法应用移植到边缘计算管理平台，实现边缘节点上容器的动态部署迁移及扩缩容。

（2）对现有的边缘计算中计算迁移算法国内外研究现状、深度强化学习以及容器编排工具 Kubernetes 进行了深入的研究，具备了一定的理论基础。

（3）本人所在项目组具备高性能服务器用于模拟边缘环境，为本课题的实现提供了可靠的硬件条件。

因此，从可行性上分析，本方案是可行的。

三、工作进度安排

（应包括文献调研，设计，研制和调试，实验，实验数据的分析处理，撰写论文等）

1. 2022 年 11 月 01 日至 2022 年 12 月 31 日：研究计算迁移、深度强化学习算法，调研深度强化学习算法的实现过程、原理，比较传统的计算迁移算法与基于深度强化学习的计算迁移算法的不同，以及它们各自的优缺点。
2. 2023 年 01 月 01 日至 2023 年 04 月 30 号：根据前面对计算迁移、深度强化学习的理论研究，以及针对迁移方案进行的数据仿真实验，开展基于深度强化学习的计算迁移研究。
3. 2023 年 05 月 01 日至 2023 年 07 月 31 日：针对计算迁移进行的数据仿真对比实验，展开基于深度强化学习的边缘计算迁移研究并发表一篇专利或论文。
4. 2023 年 08 月 01 日至 2023 年 10 月 31 日：根据第一个研究内容，不断修改完善，同时，根据对本课题内容的研究，开始实施基于内容流行度的任务请求缓存研究。
5. 2023 年 11 月 01 日至 2023 年 12 月 31 日：搭建并实现基于容器的 MEC 的边缘云计算管理平台。
6. 2024 年 01 月 01 日至 2024 年 03 月 31 日：完成毕业论文的撰写工作，作好答辩准备。

四、预期成果

1. 发表学术论文 1 篇或发明专利 1 篇；
2. 论文对应的原型系统；
3. 研究报告和论文各 1 份。

导师意见

邱华杰同学，对边缘计算环境下计算迁移算法与缓存策略进行深入研究。首先，将深度强化学习应用在计算迁移算法中，充分利用 MEC 计算资源，实现边缘计算迁移优化策略。其次，研究了一种请求任务缓存策略，在提高缓存命中率的同时可以有效减少降低任务计算迁移的成本。论文的研究工作具有很好的理论价值和应用前景。

同意开题！

导师签名：（学校）



（企业）

2022 年 11 月 03 日

年 月 日

